

All semi-local longest common subsequences in subquadratic time

Alexander Tiskin

Department of Computer Science, The University of Warwick,
Coventry CV4 7AL, United Kingdom
`tiskin@dcs.warwick.ac.uk`

Abstract. For two strings a, b of lengths m, n respectively, the longest common subsequence (LCS) problem consists in comparing a and b by computing the length of their LCS. In this paper, we define a generalisation, called “the all semi-local LCS problem”, where each string is compared against all substrings of the other string, and all prefixes of each string are compared against all suffixes of the other string. An explicit representation of the output lengths is of size $\Theta((m+n)^2)$. We show that the output can be represented implicitly by a geometric data structure of size $O(m+n)$, allowing efficient queries of the individual output lengths. The currently best all string-substring LCS algorithm by Alves et al. can be adapted to produce the output in this form. We also develop the first all semi-local LCS algorithm, running in time $o(mn)$ when m and n are reasonably close. Compared to a number of previous results, our approach presents an improvement in algorithm functionality, output representation efficiency, and/or running time.

1 Introduction

Given two strings a, b of lengths m, n respectively, the longest common subsequence (LCS) problem consists in comparing a and b by computing the length of their LCS. In this paper, we define a generalisation, called “the all semi-local LCS problem”, where each string is compared against all substrings of the other string, and all prefixes of each string are compared against all suffixes of the other string. The all semi-local LCS problem arises naturally in the context of LCS computations on substrings. It is closely related to local sequence alignment (see e.g. [7, 9]) and to approximate string matching (see e.g. [6, 15]).

A standard approach to string comparison is representing the problem as a dag (directed acyclic graph) of size $\Theta(mn)$ on an $m \times n$ grid of nodes. The basic LCS problem, as well as its many generalisations, can be solved by dynamic programming on this dag in time $O(mn)$ (see e.g. [6, 7, 15, 9]). It is well-known (see e.g. [13, 1] and references therein) that all essential information in the grid dag can in fact be represented by a data structure of size $O(m+n)$. In this paper, we expose a rather surprising (and to the best of our knowledge, previously unnoticed) connection between this linear-size representation of the string

comparison dag, and a standard computational geometry problem known as dominance counting.

If the output lengths of the all semi-local LCS problem are represented explicitly, the total size of the output is $\Theta((m+n)^2)$, corresponding to $m^2 + n^2$ possible substrings and $2mn$ possible prefix-suffix pairs. To reduce the storage requirements, we allow the output lengths to be represented implicitly by a smaller data structure that allows efficient retrieval of individual output values. Using previously known linear-size representations of the string comparison dag, retrieval of an individual output length typically requires scanning of at least a constant fraction of the representing data structure, and therefore takes time $O(m+n)$. By exploiting the geometry connection, we show that the output lengths can be represented by a set of $m+n$ grid points. Individual output lengths can be obtained from this representation by dominance counting queries. This leads to a data structure of size $O(m+n)$, that allows to query an individual output length in time $O(\frac{\log(m+n)}{\log \log(m+n)})$, using a recent result by JaJa, Mortensen and Shi [8]. The described approach presents a substantial improvement in query efficiency over previous approaches.

It has long been known [14, 5] that the (global) LCS problem can be solved in subquadratic¹ time $O(\frac{mn}{\log(m+n)})$ when m and n are reasonably close. Alves et al. [1], based on an idea of Schmidt [17], proposed an all string-substring (i.e. restricted semi-local) LCS algorithm that runs in time $O(mn)$. In this paper, we propose the first all semi-local LCS algorithm, which runs in subquadratic time $O(\frac{mn}{\log(m+n)^{1/2}})$ when m and n are reasonably close. This improves on [1] simultaneously in algorithm functionality, output representation efficiency, and running time.

2 Previous work

Although our generic definition of the all semi-local LCS problem is new, several algorithms dealing with similar problems involving multiple substring comparison have been proposed before. The standard dynamic programming approach can be regarded as comparing all prefixes of each string against all prefixes of the other string. Papers [17, 11, 1] present several variations on the theme of comparing substrings (prefixes, suffixes) of two strings. In [12, 10], the two input strings are revealed character by character. Every new character can be either appended or prepended to the input string. Therefore, the computation is performed essentially on substrings of subsequent inputs. In [13], multiple strings sharing a common substring are compared against a common target string. A common feature in many of these algorithms is the use of linear-sized string comparison dag representation, and a suitable merging procedure that “stitches together” the representations of neighbouring dag blocks to obtain a representation for the blocks’ union. As a consequence, such algorithms could be adapted to work

¹ The term “subquadratic” here and in the paper’s title refers to the case $m = n$.

with our new, potentially more efficient geometric representation, without any increase in asymptotic time or memory requirements.

3 Problem statement and notation

We consider strings of characters from a fixed finite alphabet, denoting string concatenation by juxtaposition. Given a string, we distinguish between its contiguous *substrings*, and not necessarily contiguous *subsequences*. Special cases of a substring are a *prefix* and a *suffix* of a string. For two strings $a = \alpha_1\alpha_2\ldots\alpha_m$ and $b = \beta_1\beta_2\ldots\beta_n$ of lengths m, n respectively, the *longest common subsequence (LCS) problem* consists in computing the LCS length of a and b .

We consider a generalisation of the LCS problem, which we call the *all semi-local LCS problem*. It consists in computing the LCS lengths on substrings of a and b as follows:

- the *all string-substring LCS problem*: a against every substring of b ;
- the *all prefix-suffix LCS problem*: every prefix of a against every suffix of b ;
- symmetrically, the *all substring-string LCS problem* and the *all suffix-prefix LCS problem*, defined as above but with the roles of a and b exchanged.

It turns out that by considering this combination of problems rather than each problem separately, the algorithms can be greatly simplified.

A traditional distinction, especially in computational biology, is between global (full string against full string) and local (all substrings against all substrings) comparison. Our problem lies in between, hence the term “semi-local”. Many string comparison algorithms output either a single optimal comparison score across all local comparisons, or a number of local comparison scores that are “sufficiently close” to the globally optimal. In contrast with this approach, we require to output all the locally optimal comparison scores.

In addition to standard integer indices $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$, we use *odd half-integer*² indices $\hat{\mathbb{Z}} = \{\ldots, -\frac{5}{2}, -\frac{3}{2}, -\frac{1}{2}, \frac{1}{2}, \frac{3}{2}, \frac{5}{2}, \ldots\}$. For two numbers i, j , we write $i \trianglelefteq j$ if $j - i \in \{0, 1\}$, and $i \triangleleft j$ if $j - i = 1$. We denote

$$\begin{aligned} [i : j] &= \{i, i+1, \ldots, j-1, j\} \\ \langle i : j \rangle &= \{i + \frac{1}{2}, i + \frac{3}{2}, \ldots, j - \frac{3}{2}, j - \frac{1}{2}\} \end{aligned}$$

4 Problem analysis

It is well-known that an instance of the LCS problem can be represented by a dag (directed acyclic graph) on an $m \times n$ grid of nodes, where character matches correspond to edges of weight 1, and gaps to edges of weight 0. To describe our algorithms, we need a slightly extended version of this representation, where the representing dag is embedded in an infinite grid dag.

² It would be possible to reformulate all our results using only integers. However, using odd half-integers helps to make the exposition simpler and more elegant.

Definition 1. Let $m, n \in \mathbb{N}$. A grid dag G is a weighted dag, defined on the set of nodes $v_{i,j}$, $i \in [0 : m]$, $j \in [0 : n]$. For all $i \in [1 : m]$, $j \in [1 : n]$:

- horizontal edge $v_{i,j-1} \rightarrow v_{i,j}$ and vertical edge $v_{i-1,j} \rightarrow v_{i,j}$ are both always present in G and have weight 0;
- diagonal edge $v_{i-1,j-1} \rightarrow v_{i,j}$ may or may not be present in G ; if present, it has weight 1.

Given an instance of the all semi-local LCS problem, its *corresponding grid dag* is an $m \times n$ grid dag, where the diagonal edge $v_{i-1,j-1} \rightarrow v_{i,j}$ is present, iff $\alpha_i = \beta_j$.

Common string-substring, suffix-prefix, prefix-suffix, and substring-string subsequences correspond, respectively, to paths of the following form in the grid dag:

$$v_{0,j} \rightsquigarrow v_{m,j'} \quad v_{i,0} \rightsquigarrow v_{m,j'} \quad v_{0,j} \rightsquigarrow v_{i',n} \quad v_{i,0} \rightsquigarrow v_{i',n} \quad (1)$$

where $i, i' \in [0 : m]$, $j, j' \in [0 : n]$. The length of each subsequence is equal to the weight of its corresponding path. The solution to the all semi-local LCS problem is equivalent to finding the weight of a maximum-weight path of each of the four types (1) between every possible pair of endpoints. (Since the graph is acyclic, this is also equivalent to finding the weight of the corresponding minimum-weight path in a grid dag where all the weights are negated.)

Definition 2. Given an $m \times n$ grid dag G , its extension G^+ is an infinite weighted dag, defined on the set of nodes $v_{i,j}$, $i, j \in \mathbb{Z}$ and containing G as a subgraph. For all $i, j \in \mathbb{Z}$:

- horizontal edge $v_{i,j-1} \rightarrow v_{i,j}$ and vertical edge $v_{i-1,j} \rightarrow v_{i,j}$ are both always present in G^+ and have weight 0;
- when $i \in [1 : m]$, $j \in [1 : n]$, diagonal edge $v_{i-1,j-1} \rightarrow v_{i,j}$ is present in G^+ iff it is present in G ; if present, it has weight 1;
- otherwise, diagonal edge $v_{i-1,j-1} \rightarrow v_{i,j}$ is always present in G^+ and has weight 1.

An infinite dag that is an extension of some (finite) grid dag will be called an *extended grid dag*. When dag G^+ is the extension of dag G , we will say that G is the *core* of G^+ . Relative to G^+ , we will call the nodes of G *core nodes*.

By using the extended grid dag representation, the four path types (1) can be reduced to a single type, corresponding to the all string-substring (or, symmetrically, substring-string) LCS problem on an extended set of indices.

Definition 3. Given an $m \times n$ grid dag G , its extended horizontal (respectively, vertical) score matrix is an infinite matrix defined by

$$A(i, j) = \max \text{weight}(v_{0,i} \rightsquigarrow v_{m,j}) \quad i, j \in \mathbb{Z} \quad (2)$$

$$A^*(i, j) = \max \text{weight}(v_{i,0} \rightsquigarrow v_{j,n}) \quad i, j \in \mathbb{Z} \quad (3)$$

where the maximum is taken across all paths between the given endpoints in the extension G^+ . If $i = j$, we have $A(i, j) = 0$. By convention, if $j < i$, then we let $A(i, j) = j - i < 0$.

The maximum path weights for each of the four path types (1) can be obtained from the extended horizontal score matrix (2) as follows:

$$\begin{aligned}\max weight(v_{0,j} \rightsquigarrow v_{m,j'}) &= A(j, j') \\ \max weight(v_{i,0} \rightsquigarrow v_{m,j'}) &= A(-i, j') - i \\ \max weight(v_{0,j} \rightsquigarrow v_{i',n}) &= A(j, m+n-i') - m + i' \\ \max weight(v_{i,0} \rightsquigarrow v_{i',n}) &= A(-i, m+n-i') - m - i + i'\end{aligned}$$

where $i, i' \in [0 : m]$, $j, j' \in [0 : n]$, and the maximum is taken across all paths between the given endpoints. The same maximum path weights can be obtained analogously from the extended vertical score matrix (3).

For most of this section, we will concentrate on the properties of extended horizontal score matrices, referring to them simply as “extended score matrices”. By symmetry, extended vertical score matrices will have analogous properties. We assume $i, j \in \mathbb{Z}$, unless indicated otherwise.

Theorem 1. *An extended score matrix has the following properties:*

$$A(i, j) \leq A(i-1, j); \quad (4)$$

$$A(i, j) \leq A(i, j+1); \quad (5)$$

$$\text{if } A(i, j+1) < A(i-1, j+1), \text{ then } A(i, j) < A(i-1, j); \quad (6)$$

$$\text{if } A(i-1, j) < A(i-1, j+1), \text{ then } A(i, j) < A(i, j+1). \quad (7)$$

Proof. A path $v_{0,i-1} \rightsquigarrow v_{m,j}$ can be obtained by first following a horizontal edge of weight 0: $v_{0,i-1} \rightarrow v_{0,i} \rightsquigarrow v_{m,j}$. Therefore, $A(i, j) \leq A(i-1, j)$. On the other hand, any path $v_{0,i-1} \rightsquigarrow v_{m,j}$ consists of a subpath $v_{0,i-1} \rightsquigarrow v_{l,i}$ of weight at most 1, followed by a subpath $v_{l,i} \rightsquigarrow v_{m,j}$. Therefore, $A(i, j) \geq A(i-1, j) - 1$. We thus have (4) and, by symmetry, (5).

A crossing pair of paths $v_{0,i} \rightsquigarrow v_{m,j}$ and $v_{0,i-1} \rightsquigarrow v_{m,j+1}$ can be rearranged into a non-crossing pair of paths $v_{0,i-1} \rightsquigarrow v_{m,j}$ and $v_{0,i} \rightsquigarrow v_{m,j+1}$. Therefore, we have the *Monge property*:

$$A(i, j) + A(i-1, j+1) \leq A(i-1, j) + A(i, j+1)$$

Rearranging the terms

$$A(i-1, j+1) - A(i, j+1) \leq A(i-1, j) - A(i, j)$$

and applying (4), we obtain (6) and, by symmetry, (7). $\square$

The properties of Theorem 1 are symmetric with respect to i and $n-j$. Alves et al. [1] introduce the same properties but do not make the most of their symmetry. We aim to exploit symmetry to the full.

Corollary 1. *An extended score matrix has the following properties:*

$$\begin{aligned}\text{if } A(i, j) < A(i-1, j), \text{ then } A(i, j') < A(i-1, j') \text{ for all } j' \leq j; \\ \text{if } A(i, j) = A(i-1, j), \text{ then } A(i, j') = A(i-1, j') \text{ for all } j' \geq j; \\ \text{if } A(i, j) < A(i, j+1), \text{ then } A(i', j) < A(i', j+1) \text{ for all } i' \geq i; \\ \text{if } A(i, j) = A(i, j+1), \text{ then } A(i', j) = A(i', j+1) \text{ for all } i' \leq i.\end{aligned}$$

Proof. These are the well-known properties of matrix A and its transpose A^T being *totally monotone*. In both pairs, the properties are each other's contrapositive, and follow immediately from Theorem 1. $\square$

Informally, Corollary 1 says that the inequality between the corresponding elements in two successive rows (respectively, columns) “propagates to the left (respectively, downwards)”, and the equality “propagates to the right (respectively, upwards)”. Recall that by convention, $A(i, j) = j - i$ for all index pairs $j < i$. Therefore, we always have an inequality between the corresponding elements in successive rows or columns in the lower triangular part of matrix A . If we fix i and scan the set of indices j from $j = -\infty$ to $j = +\infty$, an inequality may change to an equality at most once. We call such a value of j *critical* for i . Symmetrically, if we fix j and scan the set of indices i from $i = +\infty$ to $j = -\infty$, an inequality may change to an equality at most once, and we can identify values of i that are critical for j . Crucially, for all pairs (i, j) , index i will be critical for j if and only if index j is critical for i . This property lies at the core of our method, which is based on the following definition.

Definition 4. An odd half-integer point $(i, j) \in \hat{\mathbb{Z}}^2$ is called *A-critical*, if

$$A(i + \tfrac{1}{2}, j - \tfrac{1}{2}) < A(i - \tfrac{1}{2}, j - \tfrac{1}{2}) = A(i + \tfrac{1}{2}, j + \tfrac{1}{2}) = A(i - \tfrac{1}{2}, j + \tfrac{1}{2})$$

In particular, point (i, j) is never *A-critical* for $i > j$. When $i = j$, point (i, j) is *A-critical* iff $A(i - \tfrac{1}{2}, j + \tfrac{1}{2}) = 0$.

Corollary 2. Let $i, j \in \hat{\mathbb{Z}}^2$. For each i (respectively, j), there exists at most one j (respectively, i) such that the point (i, j) is *A-critical*.

Proof. By Corollary 1 and Definition 4. $\square$

We will represent an extended score matrix by its set of critical points. Such a representation is based on the following simple geometric concept.

Definition 5. Point (i_0, j_0) dominates³ point (i, j) , if $i_0 < i$ and $j < j_0$.

Informally, the dominated point is “below and to the left” of the dominating point in the score matrix⁴. The following theorem shows that the geometric representation of a score matrix is unique, and gives a simple formula for recovering matrix elements.

Theorem 2. For an arbitrary integer point $(i_0, j_0) \in \mathbb{Z}^2$, let $d_A(i_0, j_0)$ denote the number of (odd half-integer) *A-critical* points it dominates. We have

$$A(i_0, j_0) = j_0 - i_0 - d_A(i_0, j_0)$$

³ The standard definition of dominance requires $i < i_0$ instead of $i_0 < i$. Our definition is more convenient in the context of the LCS problem.

⁴ Note that these concepts of “below” and “left” are relative to the score matrix, and have no connection to the “vertical” and “horizontal” directions in the grid dag.

Proof. Induction on $j_0 - i_0$. Denote $d = d_A(i_0, j_0)$.

Induction base. Suppose $i_0 \geq j_0$. Then $d = 0$ and $A(i_0, j_0) = j_0 - i_0$.

Inductive step. Suppose $i_0 < j_0$. Let d' denote the number of critical points in $\langle i_0 : n \rangle \times \langle 0 : j_0 - 1 \rangle$. By the inductive hypothesis, $A(i_0, j_0 - 1) = j_0 - 1 - i_0 - d'$. We have two cases:

1. There is a critical point $(i, j_0 - \frac{1}{2})$ for some $i \in \langle i_0 : n \rangle$. Then $d = d' + 1$ and $A(i_0, j_0) = A(i_0, j_0 - 1) = j_0 - i_0 - d$ by Corollary 1.
2. There is no such critical point. Then $d = d'$ and $A(i_0, j_0) = A(i_0, j_0 - 1) + 1 = j_0 - i_0 - d$ by Corollary 1.

In both cases, the theorem statement holds for $A(i_0, j_0)$. $\square$

Recall that outside the core, the structure of an extended grid graph is trivial: all possible diagonal edges are present in the non-core subgraph. This gives rise to an additional property: when $i < -m$ or $j > m + n$, point (i, j) is A -critical iff $j - i = m$. We will call such A -critical points *trivial*. It is easy to see that an A -critical point (i, j) is non-trivial, iff either both $v_{0, i - \frac{1}{2}}$ and $v_{0, i + \frac{1}{2}}$, or both $v_{m, j - \frac{1}{2}}$ and $v_{m, j + \frac{1}{2}}$, are core nodes.

Corollary 3. *There are exactly $m + n$ non-trivial A -critical points.*

Proof. We have $A(-m, m + n) = m$. On the other hand, $A(-m, m + n) = 2m + n - d_A(-m, m + n)$ by Theorem 2. Hence, the total number of non-trivial A -critical points is $d_A(-m, m + n) = m + n$. $\square$

Since only non-trivial critical points need to be represented explicitly, Theorem 2 allows a representation of an extended score matrix by a data structure of size $O(m + n)$. There is a close connection between this representation and the canonical structure of general Monge matrices (see e.g. [4]).

Informally, Theorem 2 says that the value $A(i_0, j_0)$ is determined by the number of A -critical points dominated by (i_0, j_0) . This number can be obtained by scanning the set of all non-trivial critical points in time $O(m + n)$. However, much more efficient methods exist when preprocessing of the critical point set is allowed.

The dominance relation between points is a well-studied topic in computational geometry. The following theorems are derived from two relevant geometric results, one classical and one recent.

Theorem 3. *For an extended score matrix A , there exists a data structure which*

- *has size $O((m + n) \log(m + n))$;*
- *can be built in time $O((m + n) \log(m + n))$, given the set of all non-trivial A -critical points;*
- *allows to query an individual element of A in time $O(\log(m + n)^2)$.*

Proof. The structure in question is a 2D range tree [3] (see also [16]), built on the set of non-trivial critical points. There are $m + n$ non-trivial critical points, hence the total number of nodes in the tree is $O((m + n) \log(m + n))$. A dominance query on the set of non-trivial critical points can be answered by accessing $O(\log(m + n)^2)$ of the nodes. A dominance query on the set of trivial critical points can be answered by a simple constant-time index calculation (note that the result of such a query can only be non-zero when the query point lies outside the core subgraph of the extended grid dag). The sum of the two dominance queries above provides the total number of critical points dominated by the query point (i_0, j_0) . The value $A(i_0, j_0)$ can now be obtained by Theorem 2. $\square$

Theorem 4. *For an extended score matrix A , there exists a data structure which*

- *has size $O(m + n)$;*
- *allows to query an individual element of A in time $O(\frac{\log(m+n)}{\log \log(m+n)})$.*

Proof. As above, but the range tree is replaced by the asymptotically more efficient data structure of [8]. $\square$

While the data structure used in Theorem 4 provides better asymptotics, the range tree used in Theorem 3 is simpler, requires a less powerful computation model, and is more likely to be practical.

We conclude this section by formulating yet another previously unexploited symmetry of the all semi-local LCS problem, which will also become a key ingredient of our algorithm. This time, we consider both the horizontal score matrix A as in (2), and the vertical score matrix A^* as in (3). We show a simple one-to-one correspondence between the geometric representations of A and A^* , allowing us to switch easily between these representations.

Theorem 5. *Point (i, j) is A -critical, iff point $(-i, m + n - j)$ is A^* -critical.*

Proof. Straightforward case analysis based on Definition 4. $\square$

5 The algorithm

We now describe an efficient algorithm for the all semi-local LCS problem. We follow a divide-and-conquer approach, which refines the framework for the string-substring LCS problem developed in [17, 1].

Strings a, b are recursively partitioned into substrings. Without loss of generality, consider a partitioning $a = a_1 a_2$ into a concatenation of two substrings of length m_1, m_2 , where $m_1 + m_2 = m$. Let A, B, C denote the extended score matrices for the all semi-local LCS problems comparing respectively a_1, a_2, a against b . In every recursive call our goal is, given matrices A, B , to compute matrix C efficiently. We call this procedure *merging*. Trivially, merging can be performed in time $O((m + n)^3)$ by standard matrix multiplication over the $(\max, +)$ -semiring. By exploiting the Monge property of the matrices, the time

complexity of merging can be easily reduced to $O((m+n)^2)$, which is optimal if the matrices are represented explicitly. We show that a further reduction in the time complexity of merging is possible, by using the data representation and algorithmic ideas introduced in Section 4.

By Theorem 2, matrices A , B , C can each be represented by the sets of respectively $m_1 + n$, $m_2 + n$, $m + n$ non-trivial critical points. Alves et al. [1] use a similar representation; however, for their algorithm, n critical points per matrix are sufficient. They describe a merging procedure for the special case $m_1 = 1$ (or $m_2 = 1$), that runs in time $O(n)$. On the basis of this procedure, they develop a string-substring LCS algorithm that runs in time $O(mn)$, and produces a data structure of size $O(n)$, which can be easily converted into the critical point set for the output matrix. By adding a post-processing phase based on Theorems 3, 4, this algorithm can be adapted to produce a query-efficient output data structure.

Our new algorithm is based on a novel merging procedure, which works for arbitrary values m_1 , m_2 .

Lemma 1. *Given subproblems with extended score matrices A , B , C as described above, the sets of A - and B -critical points can be merged into the set of C -critical points in time $O(m + n^{1.5})$ and memory $O(m + n)$.*

Proof. Our goal is to compute the set of all non-trivial C -critical points. Without loss of generality, we may assume that $2m_1 = 2m_2 = m$, and that n is a power of 2 (otherwise, appropriate padding can be applied to the input). We will compute non-trivial C -critical points in two stages:

1. points $(i, k) \in \langle -m, -\frac{m}{2} \rangle \times \langle 0, \frac{m}{2} + n \rangle \cup \langle -\frac{m}{2}, n \rangle \times \langle \frac{m}{2} + n, m + n \rangle$;
2. points $(i, k) \in \langle -\frac{m}{2}, n \rangle \times \langle 0, \frac{m}{2} + n \rangle$.

It is easy to see that every non-trivial C -critical point (i, j) is computed in either the first or the second stage. Informally, each C -critical point in the first stage is obtained as a direct combination of an A -critical and a B -critical point, exactly one of which is trivial. All A -critical and B -critical points remaining in the second stage are non-trivial, and determine collectively the remaining C -critical points. However, in the second stage the direct one-to-one relationship between C -critical points and pairs of A - and B -critical points need not hold.

We now give a formal description of both stages of the algorithm.

First stage. Let $i \in \langle -m, -\frac{m}{2} \rangle$, $j = i + \frac{m}{2}$. Recall that (i, j) is a trivial A -critical point. It is straightforward to check that for all k , (i, k) is C -critical, iff (j, k) is B -critical. Therefore, all $m/2$ C -critical points in $\langle -m, -\frac{m}{2} \rangle \times \langle 0, \frac{m}{2} + n \rangle$ can be found in time $O(m + n)$. Analogously, all $m/2$ C -critical points in $\langle -\frac{m}{2}, n \rangle \times \langle \frac{m}{2} + n, m + n \rangle$ can also be found in time $O(m + n)$. The overall memory cost of the first stage is $O(m + n)$.

Second stage. First, we simplify the problem by eliminating all half-integer indices that correspond to critical points considered in the first stage. We then proceed by partitioning the resulting square index range recursively into regular

half-sized square blocks. For each block, we establish the number of C -critical points contained in it, and perform the recursive partitioning of the block as long as this number is greater than 0. (The details are omitted due to space restrictions. They will appear in the full version of the paper.)

In summary, the first stage takes time and memory $O(m+n)$. The second stage takes time and memory $O(m+n)$ for index elimination and renumbering, and then time $O(n^{1.5})$ and memory $O(n)$ for the recursion. Therefore, we have the total time and memory cost as claimed. $\square$

From now on, we assume without loss of generality that $n \leq m$. We will also assume that m and n are reasonably close, so that $(\log m)^c \leq n$ for some constant c , specified separately for each algorithm. First, we describe a simple algorithm running in overall time $O(mn)$, and then we modify it to achieve running time $o(mn)$.

Algorithm 1 (All semi-local LCS, basic version).

Input: strings a, b of length m, n , respectively; we assume $\log m \leq n \leq m$.

Output: extended score matrix on strings a, b , represented by $m+n$ non-trivial critical points.

Description. The computation proceeds recursively, partitioning the longer of the two current strings into a concatenation of two strings of equal length (within ± 1 if string length is odd). Given a current partitioning, the corresponding sets of critical points are merged by Lemma 1. Note that we now have two nested recursions: the main recursion of the algorithm, and the inner recursion of Lemma 1.

In the process of main recursion, the algorithm may (and typically will, as long as the current values of m and n are sufficiently close) alternate between partitioning string a and string b . Therefore, we will need to convert the geometric representation of a horizontal score matrix into a vertical one, and vice versa. This can be easily achieved by Theorem 5.

The base of the main recursion is $m = n = 1$.

Cost analysis. Consider the main recursion tree. The computational work in the top $\log(m/n)$ levels of the tree is at most $\log(m/n) \cdot O(m) + (m/n) \cdot O(n^{1.5}) = O(mn)$. The computational work in the remaining $2 \log n$ levels of the tree is dominated by the bottom level, which consists of $O(mn)$ instances of merging score matrices of size $O(1)$. Therefore, the total computation cost is $O(mn)$.

The main recursion tree can be evaluated depth-first, so that the overall memory cost is dominated by the top level of the main recursion, running in memory $O(n)$. $\square$

The above algorithm can now be easily modified to achieve the claimed sub-quadratic running time, using an idea originating in [2] and subsequently applied to string comparison by [14].

Algorithm 2 (All semi-local LCS, full version).

Input, output: as in Algorithm 1; we assume $(\log m)^{5/2} \leq n \leq m$.

Description. Consider an all semi-local LCS problem on strings of size $t = \frac{1}{2} \cdot \log_{\sigma} m$, where σ is the size of the alphabet. All possible instances of this problem are precomputed by Algorithm 1 (or by the algorithm of [1]). After that, the computation proceeds as in Algorithm 1. However, the main recursion is cut off at the level where block size reaches t , and the precomputed values are used as the recursion base.

Cost analysis. In the precomputation stage, there are σ^{2t} problem instances, each of which costs $O(t^2)$. Therefore, the total cost of the precomputation is $\sigma^{2t} \cdot O(t^2) = \frac{1}{4} \cdot m(\log_{\sigma} m)^2 = O\left(\frac{mn}{\log^{1/2}(m+n)}\right)$.

Consider the main recursion tree. The computational work in the top $\log(m/n)$ levels of the tree is at most $\log(m/n) \cdot O(m) + (m/n) \cdot O(n^{1.5}) = O\left(\frac{mn}{\log^{1.5} m}\right) + O\left(\frac{mn}{\log^{5/4} m}\right) = o\left(\frac{mn}{\log^{1/2}(m+n)}\right)$. The computational work in the remaining $2 \log(n/t)$ levels of the tree is dominated by the cut-off level, which consists of $O(mn/t^2)$ instances of merging score matrices of size $O(t)$. Therefore, the total computation cost is $mn/t^2 \cdot O(t^{1.5}) = O\left(\frac{mn}{t^{1/2}}\right) = O\left(\frac{mn}{\log^{1/2}(m+n)}\right)$. $\square$

6 Conclusions

We have presented a new approach to the all semi-local LCS problem. Our approach results in a significantly improved output representation, and yields the first subquadratic algorithm for the problem, with running time $O\left(\frac{mn}{\log^{1/2}(m+n)}\right)$ when m and n are reasonably close.

An immediate open question is whether the time efficiency of our algorithm can be improved even further, e.g. to match the (global) LCS algorithms of [14, 5] with running time $O\left(\frac{mn}{\log(m+n)}\right)$. Currently, our algorithm assumes constant alphabet size; it may be possible to remove this assumption by the technique of [5].

Another interesting question is whether our algorithm can be adapted to more general string comparison. The *edit distance problem* concerns a minimum-cost transformation between two strings, with given costs for character insertion, deletion and substitution. The LCS problem is equivalent to the edit distance problem with insertion/deletion cost 1 and substitution cost 2 or greater. By a constant-factor blow-up of the grid dag, our algorithm can solve the all semi-local edit distances problem, where the insertion, deletion and substitution edit costs are any constant rationals. It remains an open question whether this can be extended to arbitrary real costs, or to sequence alignment with non-linear gap penalties.

Finally, our technique appears general enough to be able to find applications beyond semi-local comparison. In particular, could it be applied in some form to the biologically important case of fully local (i.e. every substring against every substring) comparison?

7 Acknowledgement

The author is partially supported by a Royal Society Conference Grant.

References

1. C. E. R. Alves, E. N. Cáceres, and S. W. Song. An all-substrings common subsequence algorithm. *Electronic Notes in Discrete Mathematics*, 19:133–139, 2005.
2. V. L. Arlazarov, E. A. Dinic, M. A. Kronrod, and I. A. Faradzev. On economical construction of the transitive closure of an oriented graph. *Soviet Mathematical Doklady*, 11:1209–1210, 1970.
3. J. L. Bentley. Multidimensional divide-and-conquer. *Communications of the ACM*, 23(4):214–229, 1980.
4. R. E. Burkard, B. Klinz, and R. Rudolf. Perspectives of Monge properties in optimization. *Discrete Applied Mathematics*, 70:95–161, 1996.
5. M. Crochemore, G. M. Landau, and M. Ziv-Ukelson. A subquadratic sequence alignment algorithm for unrestricted score matrices. *SIAM Journal on Computing*, 32(6):1654–1673, 2003.
6. M. Crochemore and W. Rytter. *Text Algorithms*. Oxford University Press, 1994.
7. D. Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
8. J. JaJa, C. Mortensen, and Q. Shi. Space-efficient and fast algorithms for multidimensional dominance reporting and counting. In *Proceedings of the 15th ISAAC*, volume 3341 of *Lecture Notes in Computer Science*, pages 558–568, 2004.
9. N. C. Jones and P. A. Pevzner. *An introduction to bioinformatics algorithms*. Computational Molecular Biology. The MIT Press, 2004.
10. S.-R. Kim and K. Park. A dynamic edit distance table. *Journal of Discrete Algorithms*, 2:303–312, 2004.
11. G. M. Landau, E. Myers, and M. Ziv-Ukelson. Two algorithms for LCS consecutive suffix alignment. In *Proceedings of the 15th CPM*, volume 3109 of *Lecture Notes in Computer Science*, pages 173–193, 2004.
12. G. M. Landau, E. W. Myers, and J. P. Schmidt. Incremental string comparison. *SIAM Journal on Computing*, 27(2):557–582, 1998.
13. G. M. Landau and M. Ziv-Ukelson. On the common substring alignment problem. *Journal of Algorithms*, 41:338–359, 2001.
14. W. J. Masek and M. S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20:18–31, 1980.
15. G. Navarro. A guided tour to approximate string matching. *ACM Computing Surveys*, 33(1):31–88, 2001.
16. F. P. Preparata and M. I. Shamos. *Computational Geometry: An Introduction*. Texts and Monographs in Computer Science. Springer, 1985.
17. J. P. Schmidt. All highest scoring paths in weighted grid graphs and their application to finding all approximate repeats in strings. *SIAM Journal on Computing*, 27(4):972–992, 1998.